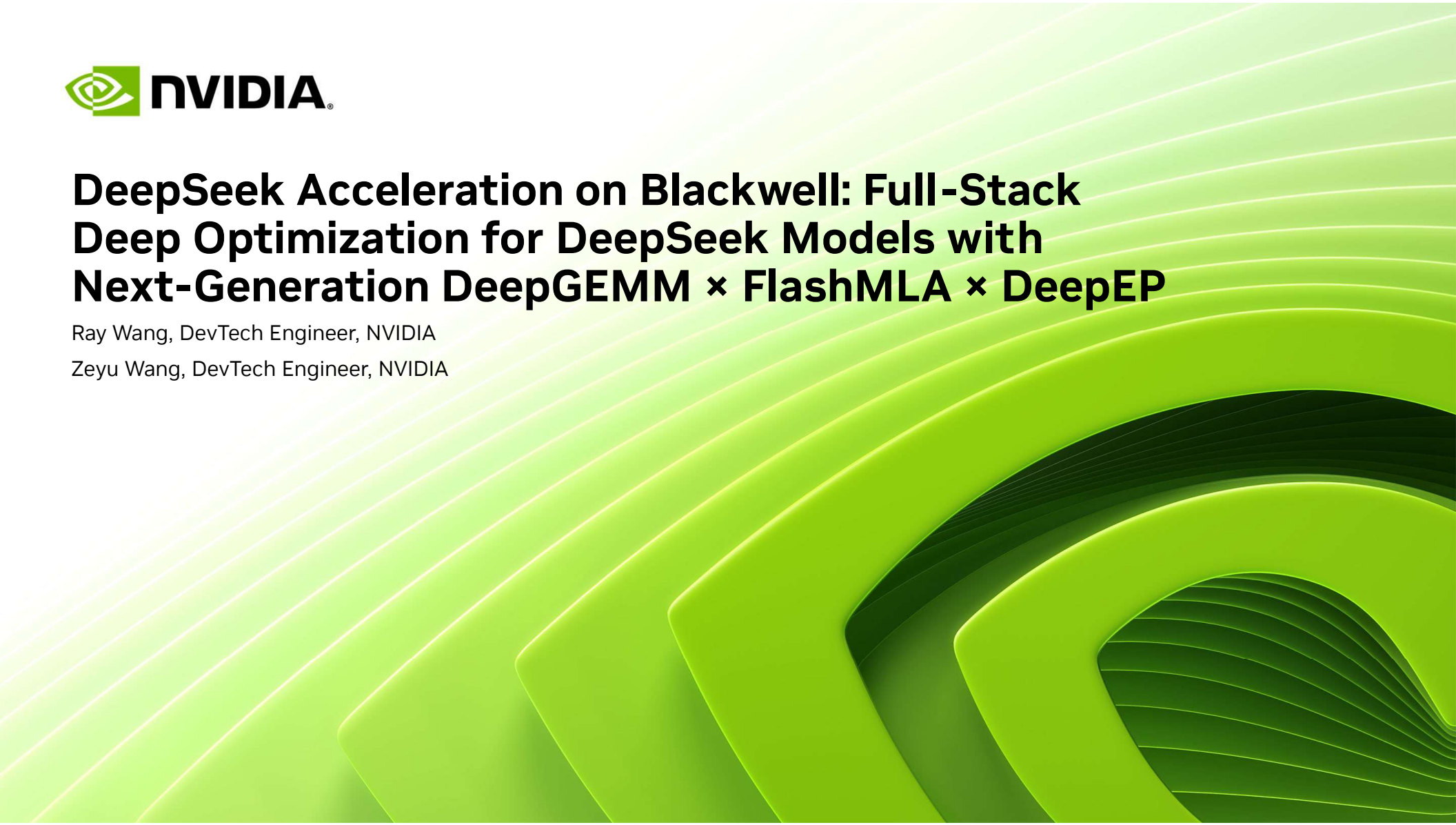


DeepSeek Acceleration on Blackwell: Full-Stack Deep Optimization for DeepSeek Models with Next-Generation DeepGEMM × FlashMLA × DeepEP

Ray Wang, DevTech Engineer, NVIDIA

Zeyu Wang, DevTech Engineer, NVIDIA

Agenda

- Part 1. DeepGEMM
- Part 2. FlashMLA
- Part 3. DeepEP

Part 1. DeepGEMM

Background

What is DeepGEMM

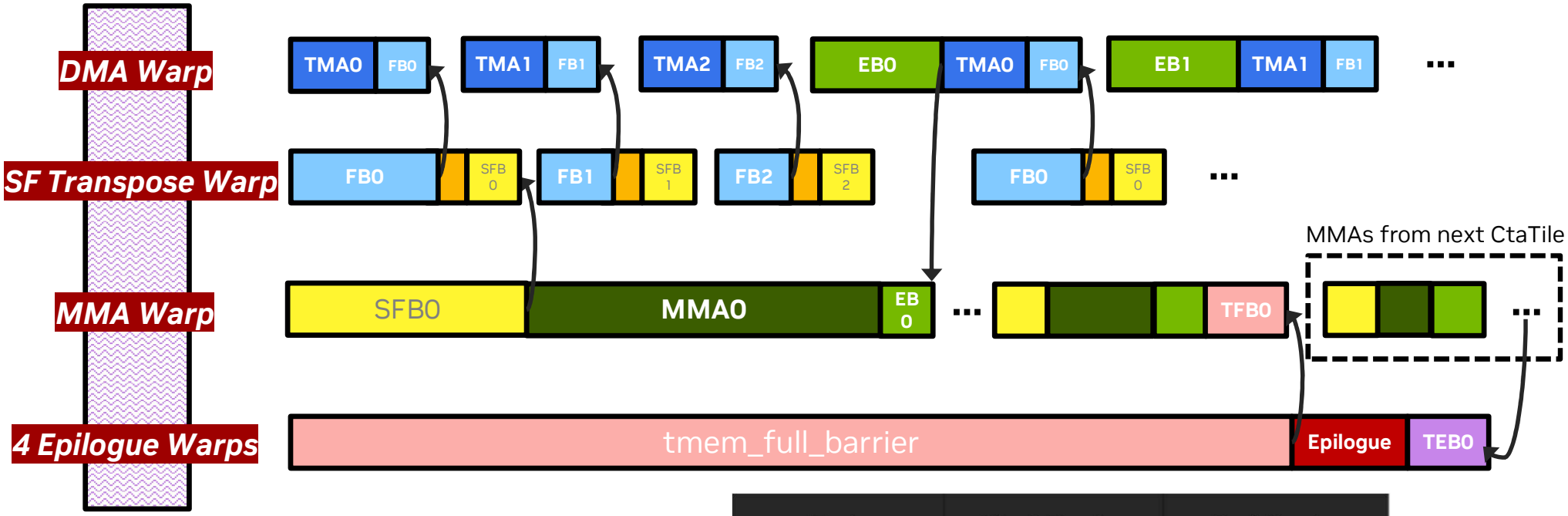
DeepGEMM is an open-source, high-performance GEMM library. It's tailored for DeepSeek model's training and inference.

- Main Functions:
  - FP8 block scale GEMM
  - FP8 block scale MoE GEMM
  - FP8 MQA logits (for DeepSeek V3.2 Indexer)
- Highlights
  - Provides a simple and out-of-the-box PyTorch interface
  - Single CUDA kernel supports both standard GEMM and MoE GEMM
  - Delivers performance on par with cuBLAS and CUTLASS
- Peak Performance (the largest GEMM in DeepSeek V3, M×N×K = 4096×7168×16384)
  - Hopper GPU: 1,556 TFLOPS (vs. 1,550 TFLOPS with cuBLAS 12.9)
  - Blackwell GPU: 3,200 TFLOPS (vs. 3,070 TFLOPS with cuBLAS 12.9)
  - cuBLAS and CUTLASS benchmarks use the non-block-scale FP8 GEMM implementation with the same script

*For technical disscussion and reference only, perf. may vary based on different product portofolio.

Blackwell Implementation

	Ada	Hopper	Blackwell (SM100)
Data Load	cp.async	cp.async.bulk (TMA)	cp.async.bulk (TMA)
FP8 MMA	mma.m16n8k32	wgmma.mma_async (*m64nNk32*)	tcgen05.mma (*m128nNk32*, 2CTA)
Accumulator Location	Registers	Registers	Tensor memory (shared across warpgroup)
Scaling	CUDA Core	CUDA Core (Overlap with MMA)	Fused with mma



Task	Start Barrier	End Barrier
TMA	Empty Bar	Full Bar
SF Transpose	Full Bar	Scaling Factor Full Bar
MMA	Scaling Factor Full Bar	Empty Bar
Mainloop	Tensor Memory Empty Bar	Tensor Memory FULL Bar
Epilogue	Tensor Memory FULL Bar	Tensor Memory Empty Bar

Each task is synchronized by a pair of barriers:

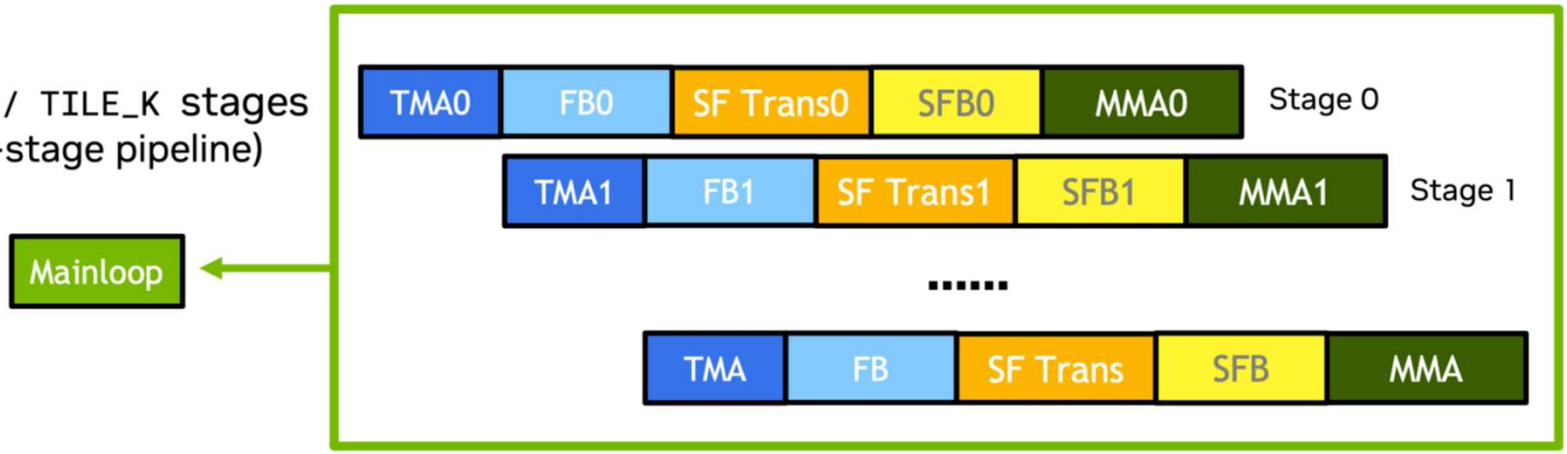
- A start-barrier that unlocks the task to begin execution
- An end-barrier that signals when the task has finished execution

Blackwell GEMM Implementation

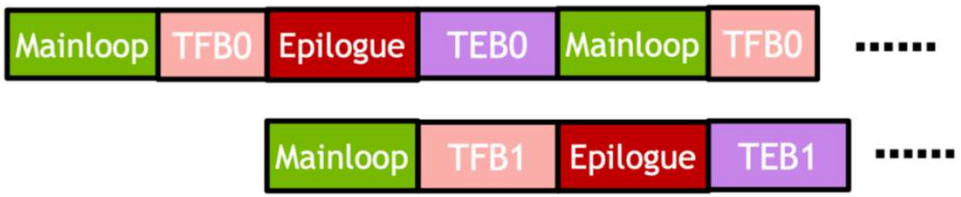
Overlap Mainloop with Epilogue

Two kinds of pipeline in DeepGEMM

Shape_K / TILE_K stages
(multi-stage pipeline)



2 stages epilogue pipeline



Blackwell GEMM Implementation

MoE GEMM

- **Scheduler maps blocks to (group, m_block, n_block)**
- Scheduler<kGemmType, ...> performs persistent scheduling inside get_next_block().
- Normal GEMM, MGroupedContiguous, and MGroupedMasked all share the same kernel logic; they only differ in how get_global_idx provides tile indices.

- **MGroupedContiguous**
  - Treats total M as one flattened continuous space when scheduling blocks.
  - Reads group id via grouped_layout[m_block_idx * BLOCK_M] and uses it to compute group-specific offsets.
  - In other words: the compute grid is scheduled on flattened M, while memory accesses are redirected to the corresponding group slice of B / scaling factors (SF).
- **MGroupedMasked**
  - Uses grouped_layout[g] = masked_m[g].
  - The scheduler iterates over groups and allocates blocks per group using ceil(masked_m[g] / BLOCK_M).
  - It automatically skips out-of-range rows beyond valid m, so no explicit input reordering is required.

Blackwell GEMM Implementation

A TN GEMM Optimization Study

Shape: m=4096, n=7168, k=2048. **Kernel:** TN GEMM for Wgrad

Step 1: Hypothesize the bottleneck: in TN GEMM the K dimension is noncontiguous in memory, which reduces read efficiency.

Step 2: Validate the hypothesis: remove all code except memory reads and measure memory throughput; a large gap from hardware peak is observed.

Step 3: Propose an optimization: increase the granularity of contiguous memory reads, i.e., choose M- and N-dimension tile sizes that are preferably multiples of 128.

- BLOCK_M and BLOCK_N both 256: shared memory is insufficient, and the number of stages is very small. ❌
- BLOCK_M = 256 and BLOCK_N smaller: BLOCK_N is either too small (128, so in a paired CTA each can only read 64 bytes, reducing efficiency) or still exceeds shared memory. ❌
- BLOCK_M = 128 and BLOCK_N = 256: in this case both A and B are read from memory at 128-byte granularity. ✅

Step 4: Verify the optimization : test memory reads in isolation; with BLOCK_M=128 and BLOCK_N=256, memory read speed improves.

Step 5: Implement the optimization: add BLOCK_N=256 support to DeepGEMM.

- Requires more than 512 Tensor Memory columns (512 columns to store results plus additional columns for scaling factors), which is not feasible on Blackwell.
- Read from memory and feed MMA with BLOCK_N=256, but when MMA writes back to Tensor Memory, only use the first 240 columns.
- TMEM layout: | Out 1 | Out 2 | SF A | SF B | == | 240 | 240 | 4 | 8 |. Because UMMA_N = 256, Out2 may be clobbered in TMEM by MMA during writeback, so we wait until Out2 has written back 16 columns before releasing Out1.

Optimization result: 1450 TFLOPS -> 1710 TFLOPS

Blackwell GEMM Benchmark

(m, n, k)	DeepGEMM	cuBLAS	CUTLASS
(128, 2112, 7168)	357	437	382
(128, 24576, 1536)	786	777	792
(128, 7168, 16384)	810	929	981
(128, 4096, 7168)	598	709	642
(128, 7168, 2048)	467	519	508
(4096, 2112, 7168)	2853	2802	2243
(4096, 24576, 1536)	2979	3203	2521
(4096, 7168, 16384)	3200	3070	2780
(4096, 4096, 7168)	2911	2959	2719
(4096, 7168, 2048)	2856	2965	2383

All units are TFLOPS

*For technical disscussion and reference only, perf. may vary based on different product portofolio.

DeepGEMM uses 1x128 by 128x128 block scale GEMM

cuBLAS and CUTLASS use the **non-block-scale** FP8 GEMM implementation with the same script

cuBLAS called by torch._scaled_mm, CUTLASS called by vllm.model_executor.layers.quantization.utils.fp8_utils.cutlass_scaled_mm

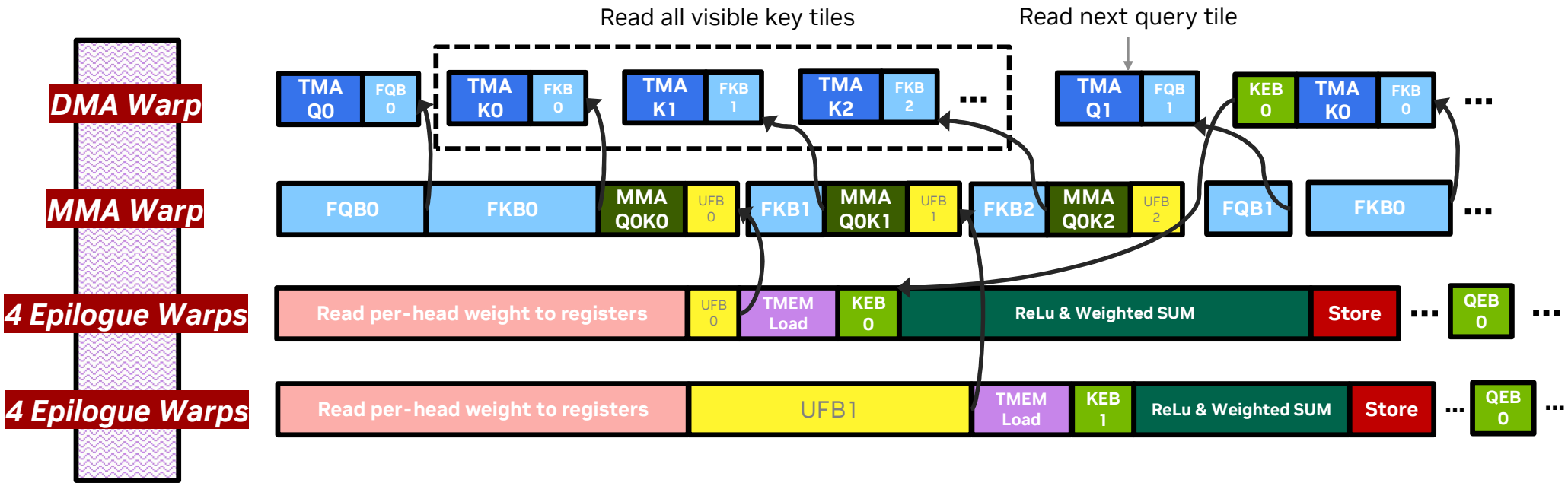
Blackwell DSA Indexer

Core of DeepSeek V3.2

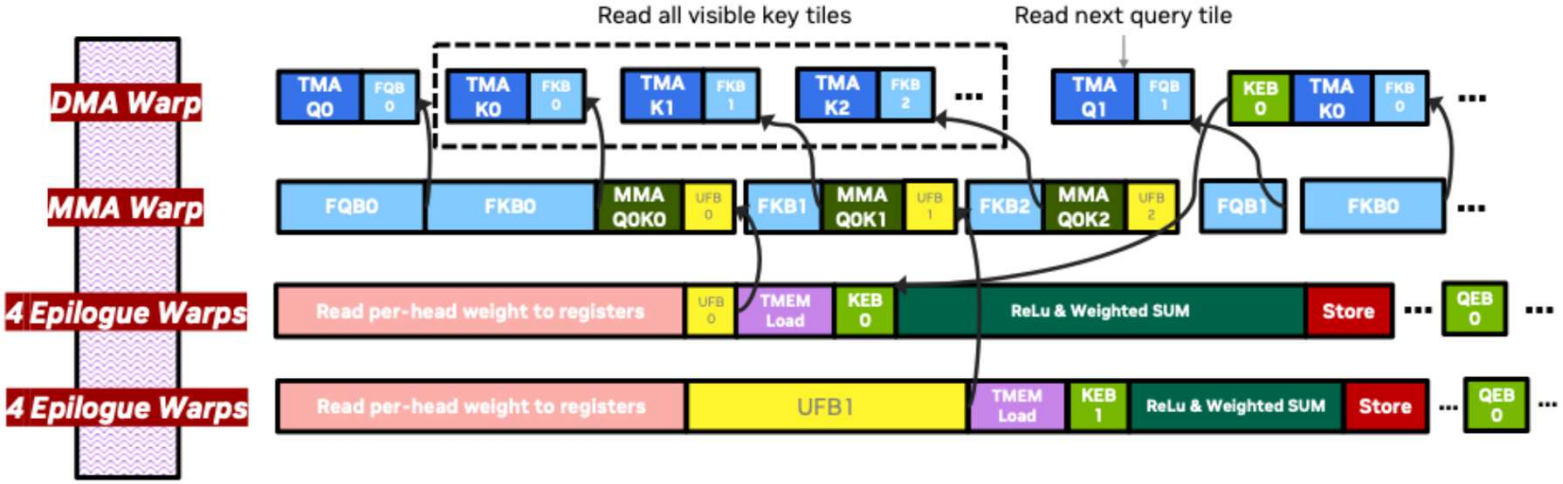
- DSA (DeepSeek Sparse Attention) is an attention mechanism designed to improve efficiency in long-context settings
- DSA Workflow: Scoring (MQA logits) → Select top-K relevant tokens → Apply sparse attention
- As context length increases, the bottleneck shifts toward the indexer, so optimizing indexer is key.
- Indexer implementation:
  - score = torch.einsum('mhd,nd->hmn', q, k) # per-head score computation
  - score = (score.relu() * weights.unsqueeze(-1).transpose(0, 1)).sum(dim=0) # Weighted sum
    - It applies an element-wise ReLU to each head's score, scales it by the corresponding per-head weight (broadcast over keys), and then sums across heads (dim=0) to produce the final scores.
  - The scores are masked here to restrict each query token to attending only to KV tokens within a specified range.
  - The scores are used for top-K selection.
- The Blackwell (SM100) GPU has fifth-generation Tensor Cores, which provide approximately twice the throughput of Hopper. In addition, SM100 introduces new features—**tensor memory** and **FFMA2**—that can effectively accelerate the weighted-sum operation.

Blackwell MQA Scores Implementation

MQA Scores Pipeline in Prefill Phase



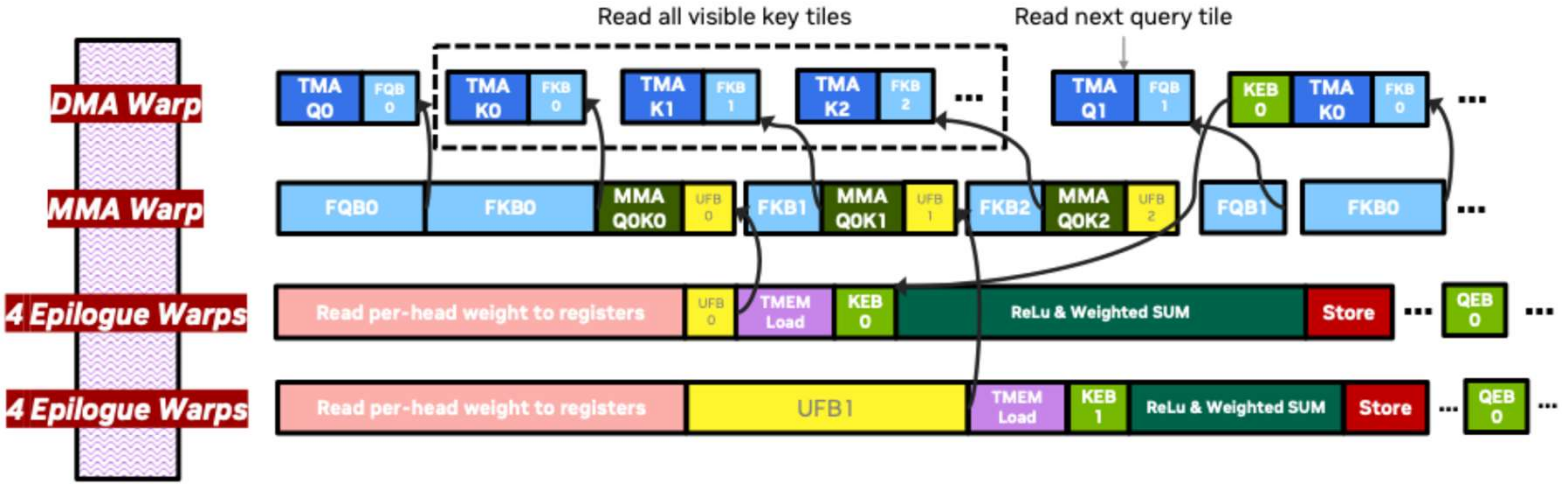
- Use two warpgroups to perform ReLU + weighted sum, better hiding memory and instruction latency.
- Each query head has a weight that is loaded at the start of the epilogue warpgroup.
- The epilogue of this kernel is heavy and is a key target for optimization.



Compared with Hopper, Blackwell (SM100) has the following advantages:

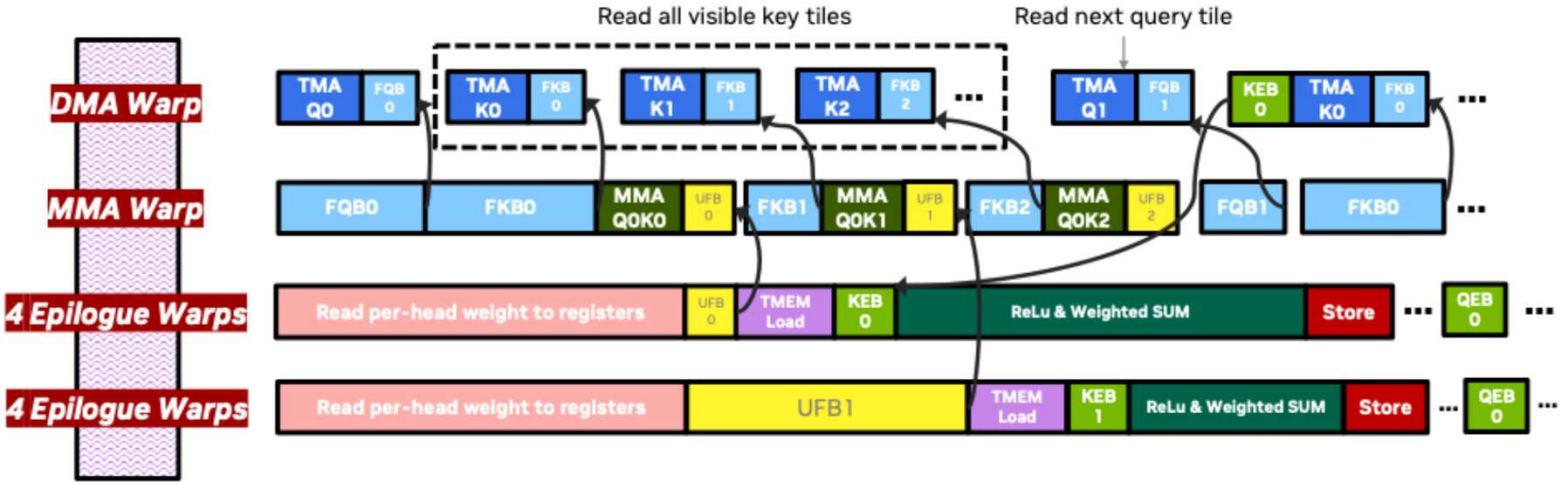
1. Blackwell's tcgen05.mma instructions deliver 2× throughput.
   - Blackwell uses *m128n128k32* MMA, K feeds operand A, Q feeds operand B.
   - Blackwell needs 2 * kNumHeads registers (128 registers) and 128 registers for MMA results.
     - `constexpr uint32_t kNumWeightsInReg = cute::min(52, kNumHeads);`

       `float weights[2][kNumWeightsInReg];`
   - DeepGEMM only preloads a portion of the weights; the remaining weights are loaded from shared memory while computing the weighted sum.
2. Blackwell writes MMA results to tensor memory, allowing the barrier to be released early.
3. Blackwell outputs the per-head result into the same thread, reducing the overhead of the weighted sum.
4. Blackwell supports FFMA2 instructions, which helps relieve instruction-issue pressure during the epilogue.



Compared with Hopper, Blackwell (SM100) has the following advantages:

1. Blackwell's tcgen05.mma instructions deliver 2× throughput.
2. Blackwell writes MMA results to tensor memory, allowing the barrier to be released early.
   - After the MMA computation, Blackwell loads (TMEM Load) the results from tensor memory into registers and can release the barrier as soon as the load completes, whereas Hopper can only release the barrier after the "ReLU + weighted-sum" finishes.
3. Blackwell outputs the per-head result into the same thread, reducing the overhead of the weighted sum.
4. Blackwell supports FFMA2 instructions, which helps relieve instruction-issue pressure during the epilogue.



Compared with Hopper, Blackwell (SM100) has the following advantages:

1. Blackwell's tcgen05.mma instructions deliver 2× throughput.
2. Blackwell writes MMA results to tensor memory, allowing the barrier to be released early.
3. Blackwell outputs the per-head result into the same thread, reducing the overhead of the weighted sum.
   - https://docs.nvidia.com/cuda/parallel-thread-execution/#tcgen05-instructions-tcgen05-ld
   - https://docs.nvidia.com/cuda/parallel-thread-execution/_images/wgmma-64N8-D.png
   - Hopper requires inter-thread shuffles to perform the weighted sum reduction.
4. Blackwell supports FFMA2 instructions, which helps relieve instruction-issue pressure during the epilogue.

Blackwell FP8 Prefill MQA Scores Benchmark

(S, SKV, H, D)	B200 (Hopper-style)	B200
(2048, 8192, 64, 128)	1510	1720
(2048, 16384, 64, 128)	1603	1804
(2048, 32768, 64, 128)	1683	1886
(4096, 8192, 64, 128)	1636	1867
(4096, 16384, 64, 128)	1757	1987
(4096, 32768, 64, 128)	1854	2080
(8192, 8192, 64, 128)	1685	1930
(8192, 16384, 64, 128)	1814	2048
(8192, 32768, 64, 128)	1856	2116

All units are TFLOPS

*For technical disscussion and reference only, perf. may vary based on different product portofolio.

- “Hopper-style” means we’re running on a B200, but the pipeline is largely the same as on Hopper.
- Compute utilization may look low, but the bottleneck is on the CUDA cores, and they’re already close to fully utilized.
- The decode and prefill versions can share the same pipeline optimizations; the main difference is the K-tile scheduling logic.

Part 2. FlashMLA

Background

FlashMLA

- FlashMLA is an open-source, high-performance attention kernel library. It is specifically designed for the Multi-head Latent Attention (MLA) architecture used in DeepSeek-V3 and DeepSeek-R1 inference.
- Main Functions
  - MLA Decoding Kernels: Improved for the unique latent vector structure of DeepSeek models.
  - Blackwell MLA Kernels: High-performance Forward and Backward kernels tailored for the next-generation NVIDIA Blackwell platform
  - DSA Prefill and Decoding Kernels: Specialized Prefill and Decoding kernels for DeepSeek Sparse Attention (DSA), supporting DeepSeek-V3.2
- Highlights
  - High Efficiency: Focuses on the decoding stage to significantly reduce inference latency.
  - Native Blackwell Support: Uses innovative CUDA features to unlock the full potential of Blackwell-series GPUs.
  - Out-of-the-box Integration: Provides a simple PyTorch interface for seamless deployment within existing LLM frameworks.

DeepSeek V3.2

New “Experimental” Model which Boosting Long-Context Efficiency

Benchmark		DeepSeek-V3.1-Terminus	DeepSeek-V3.2-Exp
General	MMLU-Pro	85.0	85.0
	GPQA-Diamond	80.7	79.9
	Humanity's Last Exam	21.7	19.8
Search Agent	BrowseComp	38.5	40.1
	BrowseComp-zh	45.0	47.9
	SimpleQA	96.8	97.1
Code	LiveCodeBench	74.9	74.1
	Codeforces-Div1	2046	2121
	Aider-Polyglot	76.1	74.5
Code Agent	SWE Verified	68.4	67.8
	SWE-bench Multilingual	57.8	57.9
	Terminal-bench	36.7	37.7
Math	AIME 2025	88.4	89.3
	HMMT 2025	86.1	83.6

DeepSeek-V3.2-Exp API

Input :

~~$0.07~~ **$0.028** cache hit

~~$0.56~~ **$0.28** cache miss

Output:

~~$1.68~~ **$0.42**

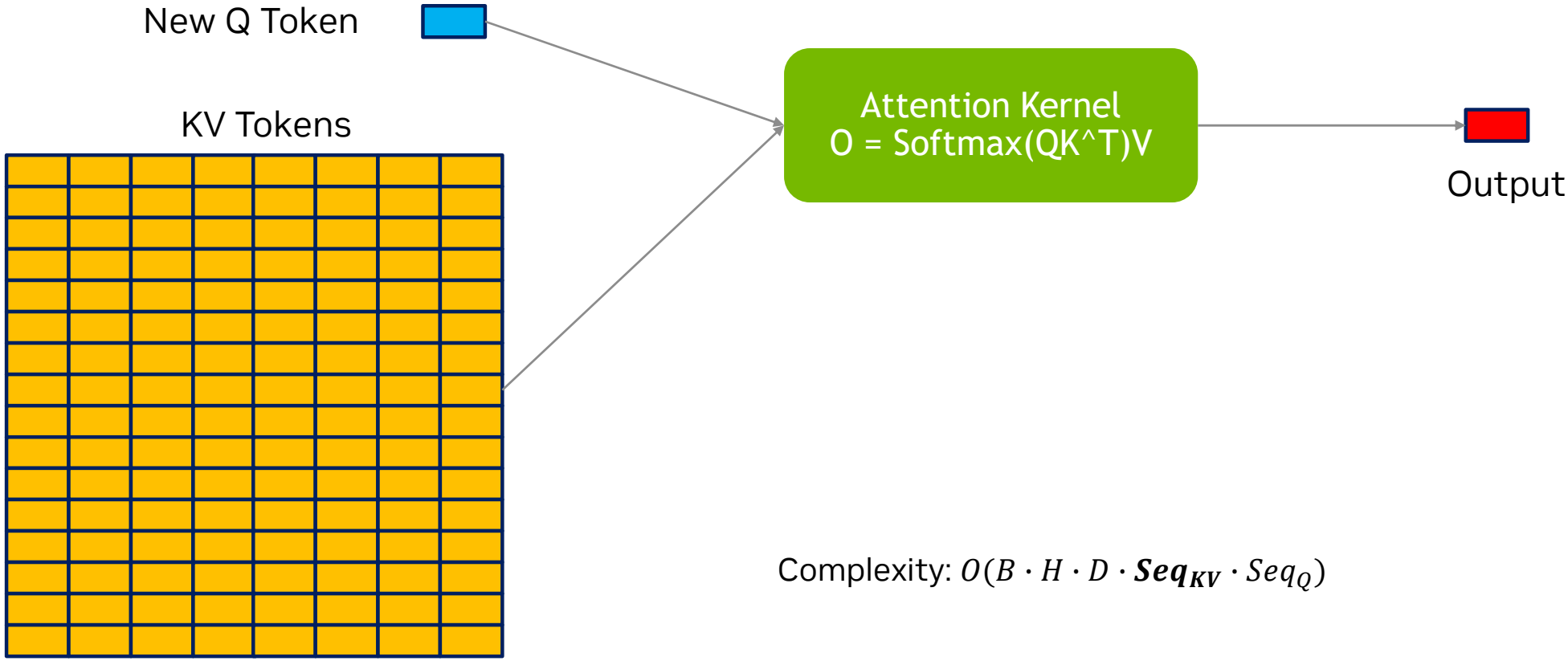
Note: Prices are quoted as $ / 1M tokens
Effective from 10:00 UTC on September 29, 2025

Sparse Attention!

Ref :Introducing DeepSeek-V3.2-Exp | DeepSeek API Docs

DSA: DeepSeek Sparse Attention

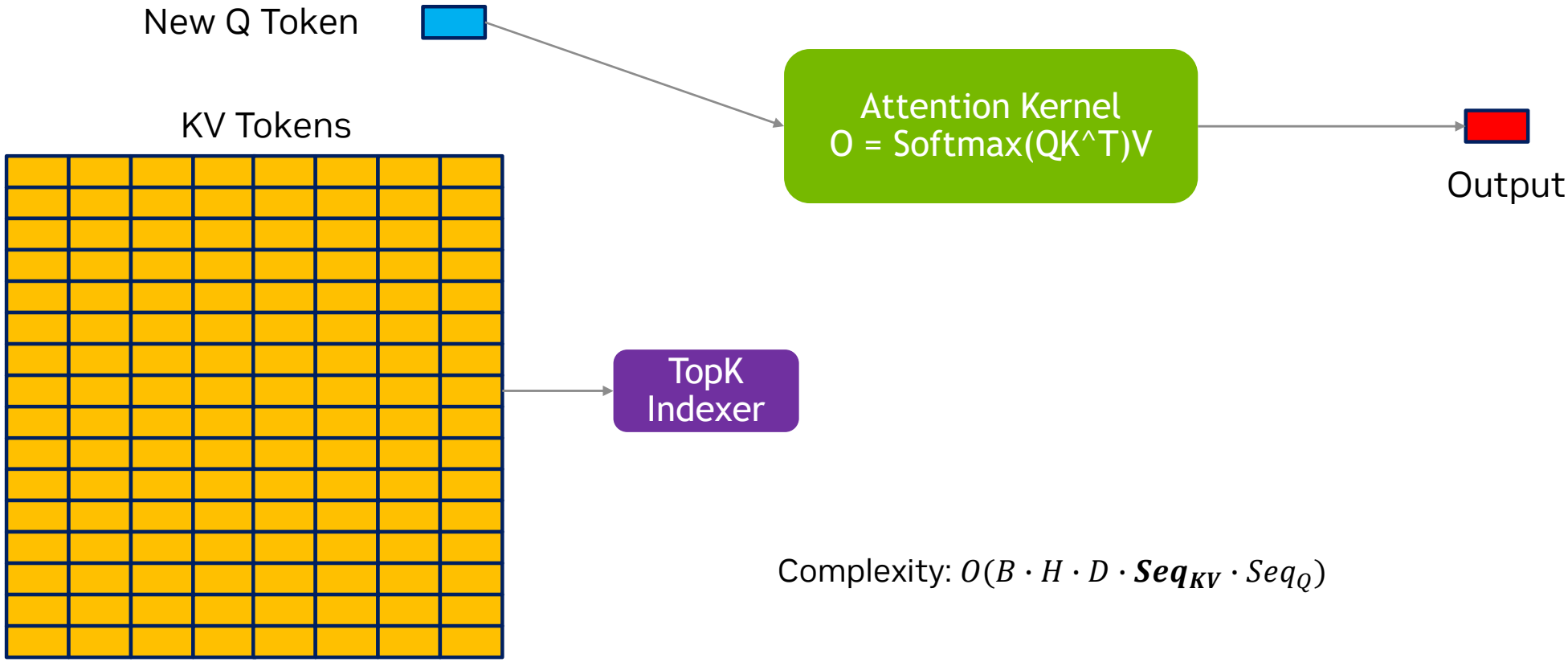
Select TopK KV Tokens during Inference



Complexity: $O(B \cdot H \cdot D \cdot \boldsymbol{Seq_{KV}} \cdot Seq_Q)$

DSA: DeepSeek Sparse Attention

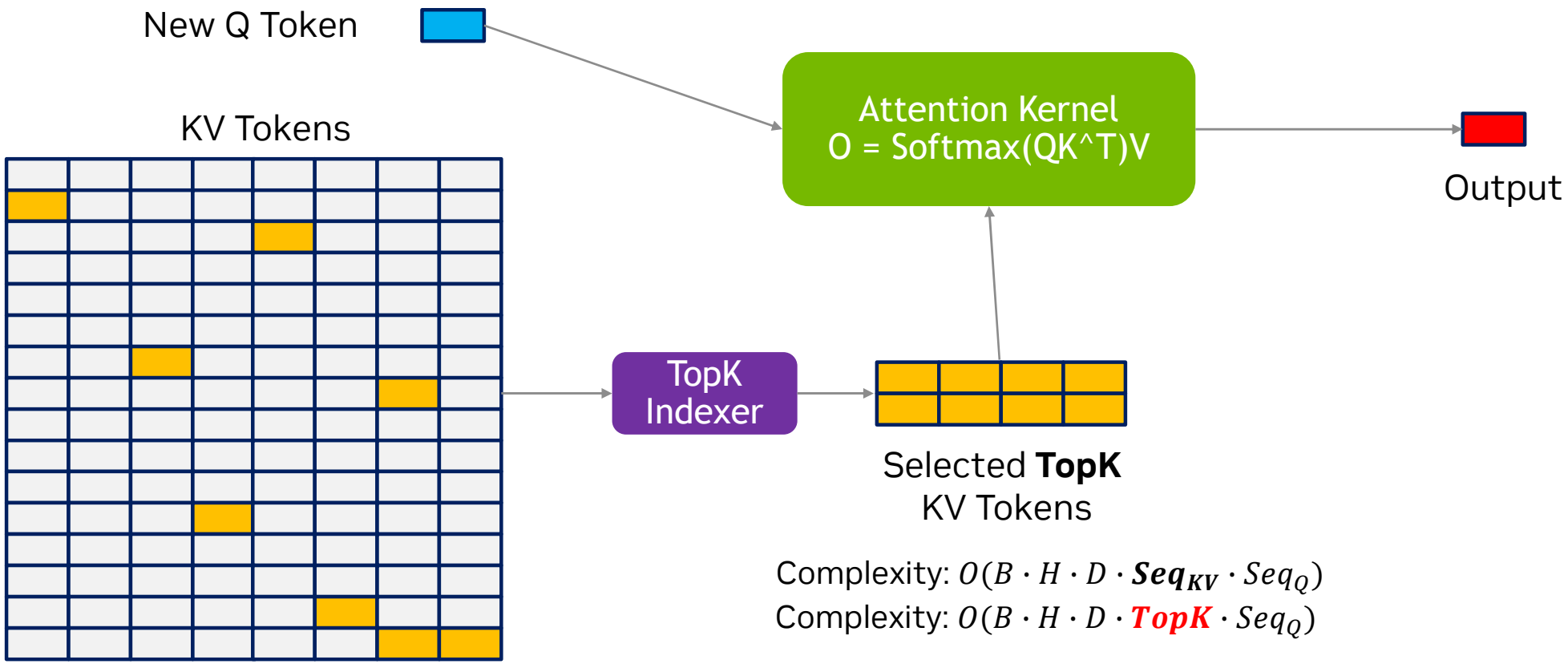
Select TopK KV Tokens during Inference



Complexity: $O(B \cdot H \cdot D \cdot \boldsymbol{Seq_{KV}} \cdot Seq_Q)$

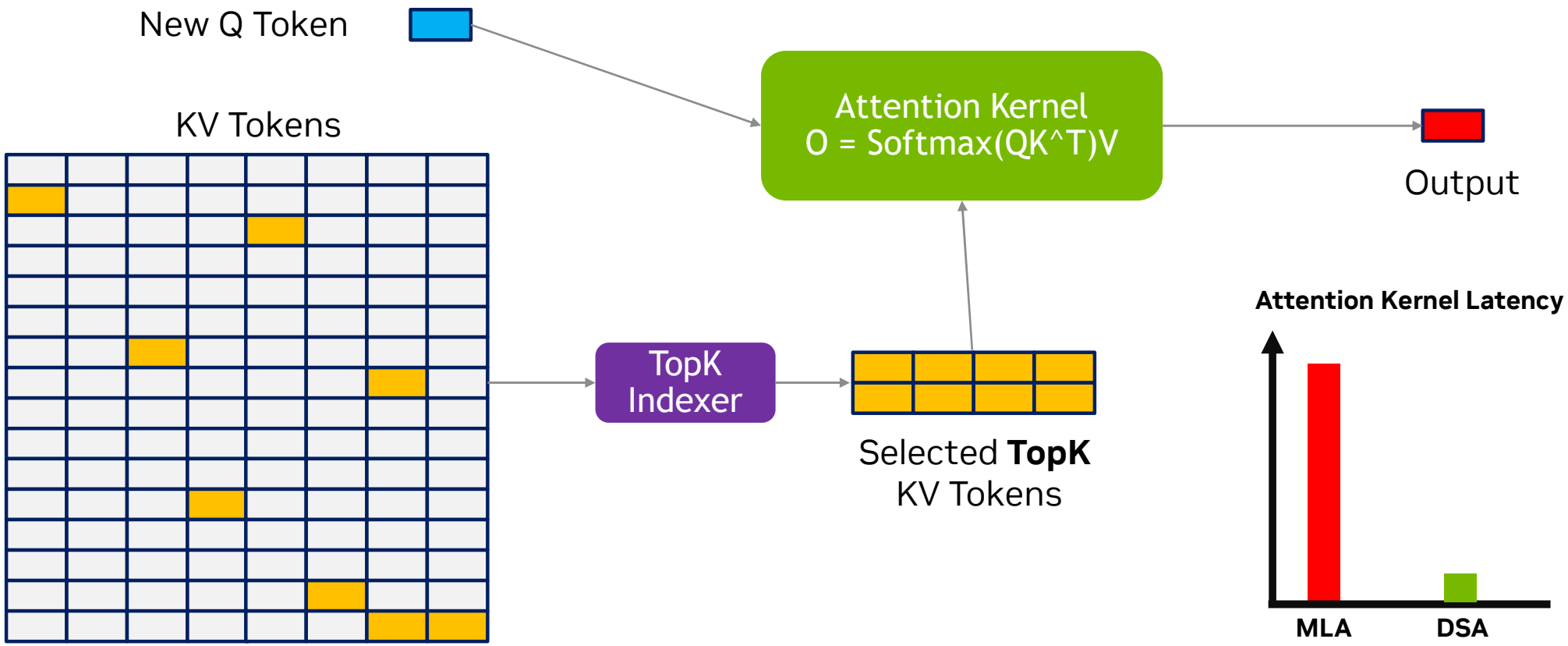
DSA: DeepSeek Sparse Attention

Select TopK KV Tokens during Inference



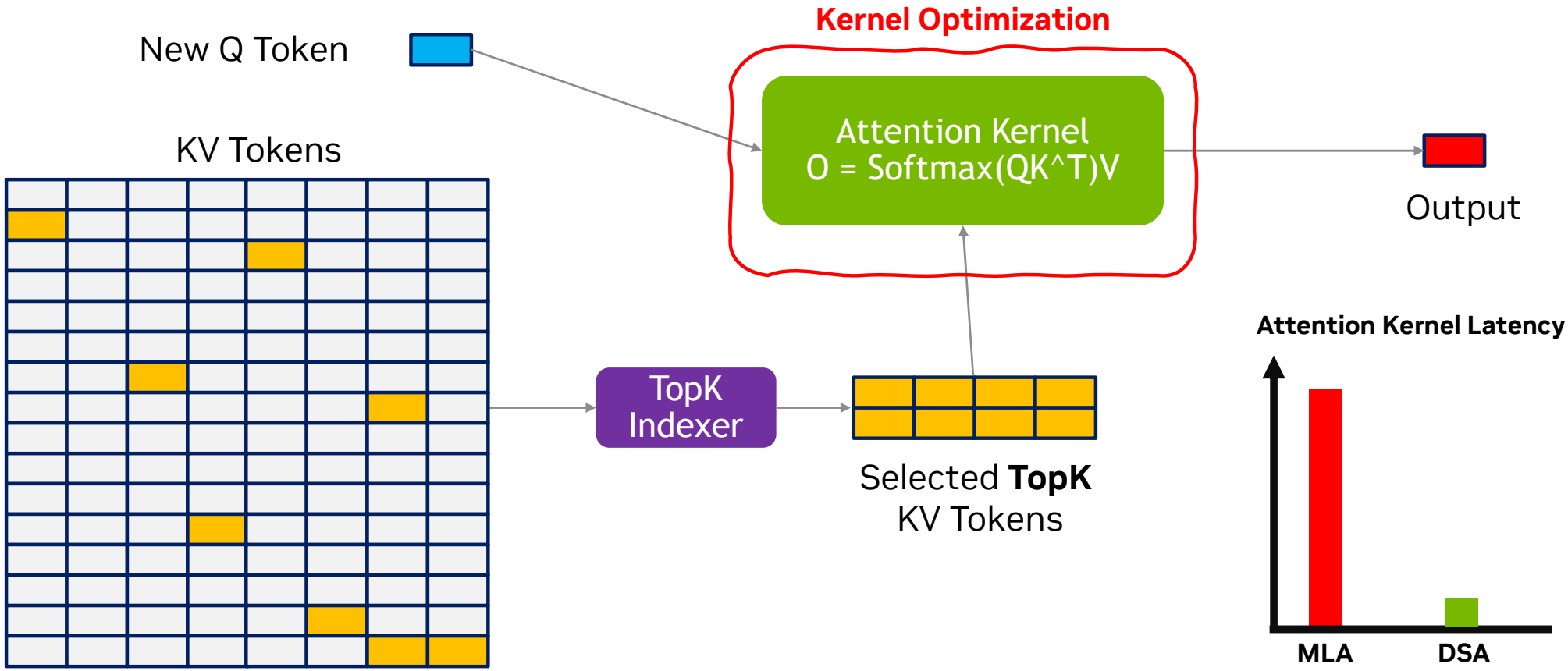
DSA: DeepSeek Sparse Attention

Select TopK KV Tokens during Inference



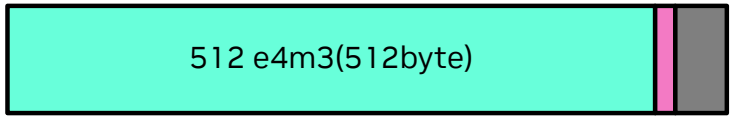
DSA: DeepSeek Sparse Attention

Select TopK KV Tokens during Inference



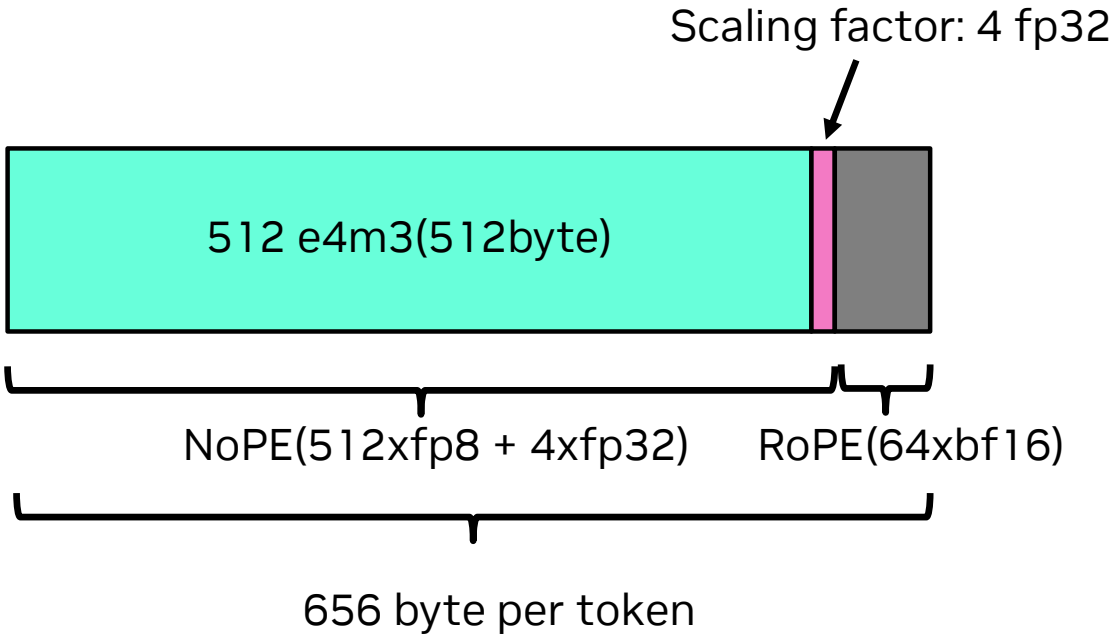
Extra Challenges of DSA

- **Sparsity** is difficult to handle by TMA
  - Waste of BW
- In Prefill: MHA is bad in **long-context** scenario
  - MHA approach: Custom Mask
  - Different Q tokens have **different topK KV** tokens
  - Seq len = 64K, topK = 2K
    - Only 4 tokens is selected in each 128 KV TILE for each Q
  - Therefore, sparse MLA kernel is necessary for prefill
- In Decoding:
  - Dequantization of FP8
    - High CUDA Core Pressure



DSA: DeepSeek Sparse Attention

FP8 KV Cache Layout



All Attention kernels are still computed in BF16

Blackwell Platform Overview

Advantages & Opportunities

- Tensor Memory
  - Replace regs of hopper to store matrix_A & matrix_O
  - 256KB
  - Data Movement
    - With REG: ***tcgen05.ld/st***
    - With SMEM: ***tcgen05.cp***

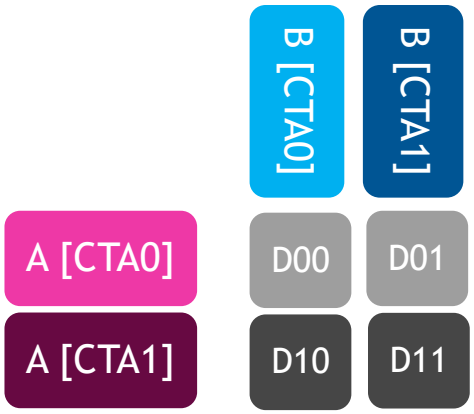
Tensor Memory Layout and Addressing

Blackwell Platform Overview

Advantages & Opportunities

- Tensor Memory
  - Replace regs of hopper to store mat_a & result
  - 256KB
  - Data Movement
    - With REG: ***tcgen05.ld/st***
    - With SMEM: ***tcgen05.cp***

- 2-CTA Tensorcore GEMM
  - Reduce SMEM BW
  - Launch by one warp of one cluster



Blackwell Platform Overview

Advantages & Opportunities

- Tensor Memory
  - Replace regs of hopper to store mat_a & result
  - 256KB
  - Data Movement
    - With REG: ***tcgen05.ld/st***
    - With SMEM: ***tcgen05.cp***
- 2-CTA Tensorcore GEMM
  - Reduce SMEM BW
- TMA.Gather4
  - Sparse TMA
  - Extra Input: **int4 idx**

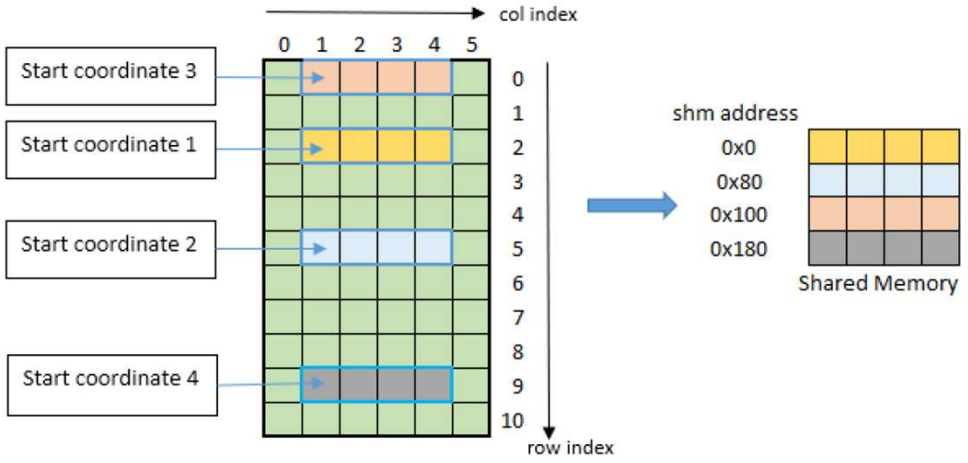


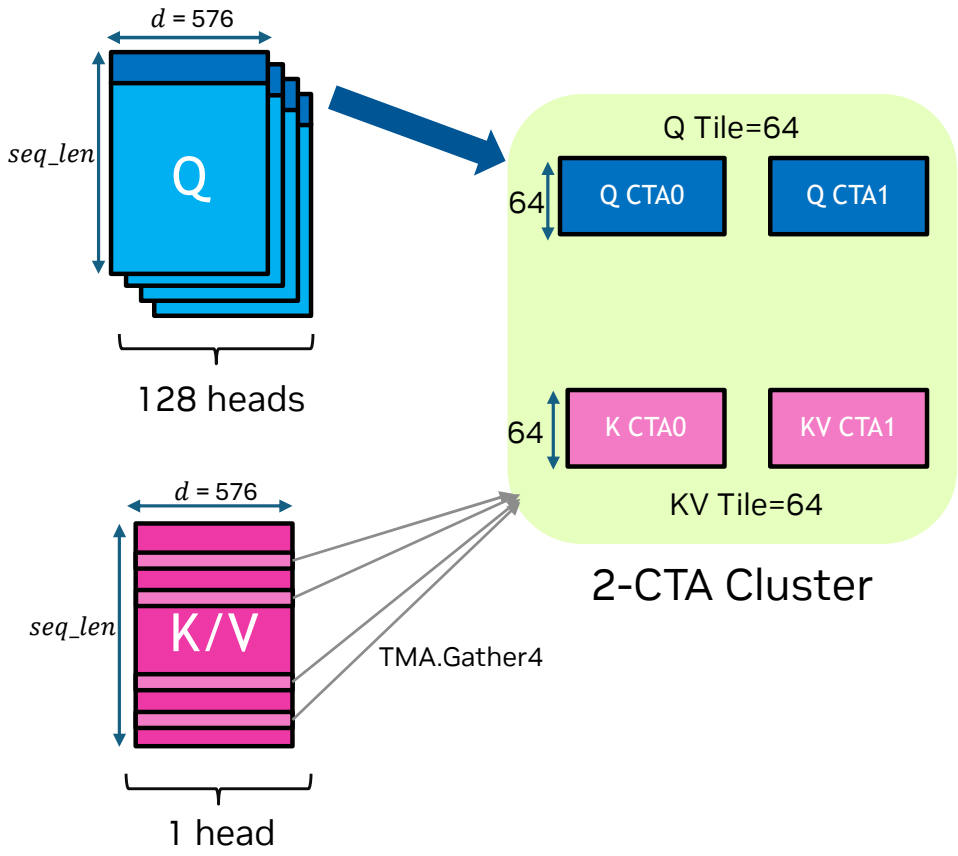
Figure 10: tiled::scatter4/tiled::gather4 mode bounding box example

DSA Sparse Prefill

Overview

- QKV config:
  - Q: [seq_len, **128**, 576]
  - K: [seq_len, 576]
  - V: [seq_len, 512] == K[:, **0: 512**]
- Kernel Setup
  - CTA dim:[2*q_len, 1, 1]
  - 2-CTA Cluster
    - 128 heads of one Q token, 64 in each CTA
    - Share same topK KV tokens

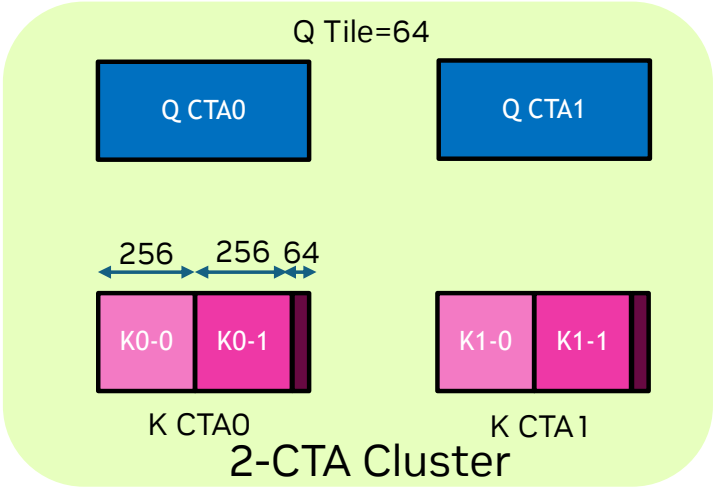
How to handle V in 2-SM GEMM?

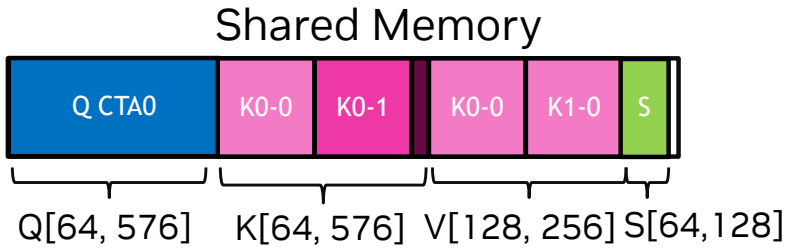


DSA Sparse Prefill

Memory Layout

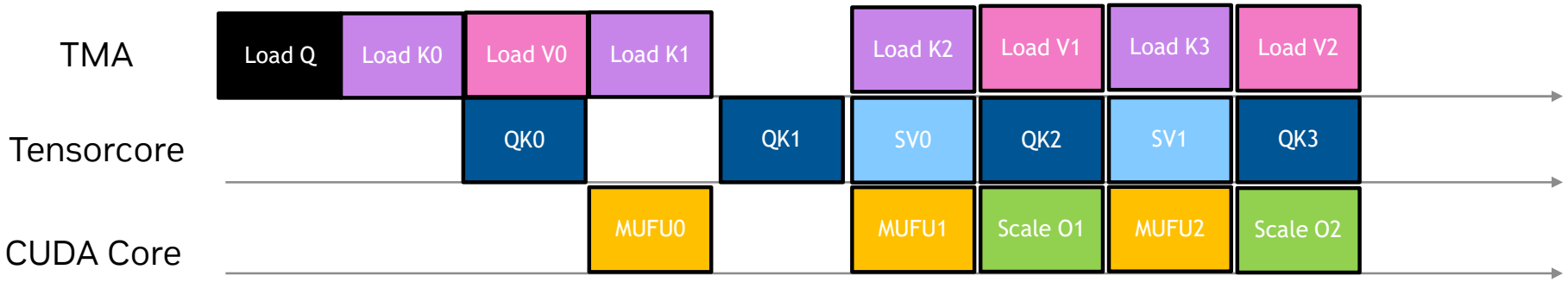
- Shared Memory:
  - Both Store K and V (Benefit from L2 Cache)
    - K Shape: [64, 576] 72KB
    - V Shape: [2*64, 256] 64KB
    - Q Shape: [64, 576] 72KB
    - S Shape: [64, 128] 16KB
    - **224** KB in Total (Limit 228KB)
  - Final Output: [64, 512] BF16 (Reuse)
- Tensor Memory:
  - P = QK^T (64 * 128 FP32 = 32KB)
  - O = SV (64 * 512 FP32 = 128KB)
  - Total: 160KB





DSA Sparse Prefill

Pipeline & Benchmark



- 512 threads(4 Warpgroup)
  - 128 for TMA-K, 128 for TMA-V
  - 128 for softmax & re-scale
  - 32 for MMA, 32 for KV topK Index

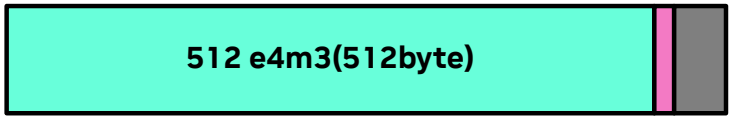
- Benchmark:
  - **1395** TFLOPS(s_q=4096, s_kv=8192, topk=2048, B200)
  - **70%** SoL(2PFLOPS in spec)

*For technical disscussion and reference only, perf. may vary based on different product portofolio.

Challenge of DSA Sparse Decoding

FP8 Layout and Dequantization

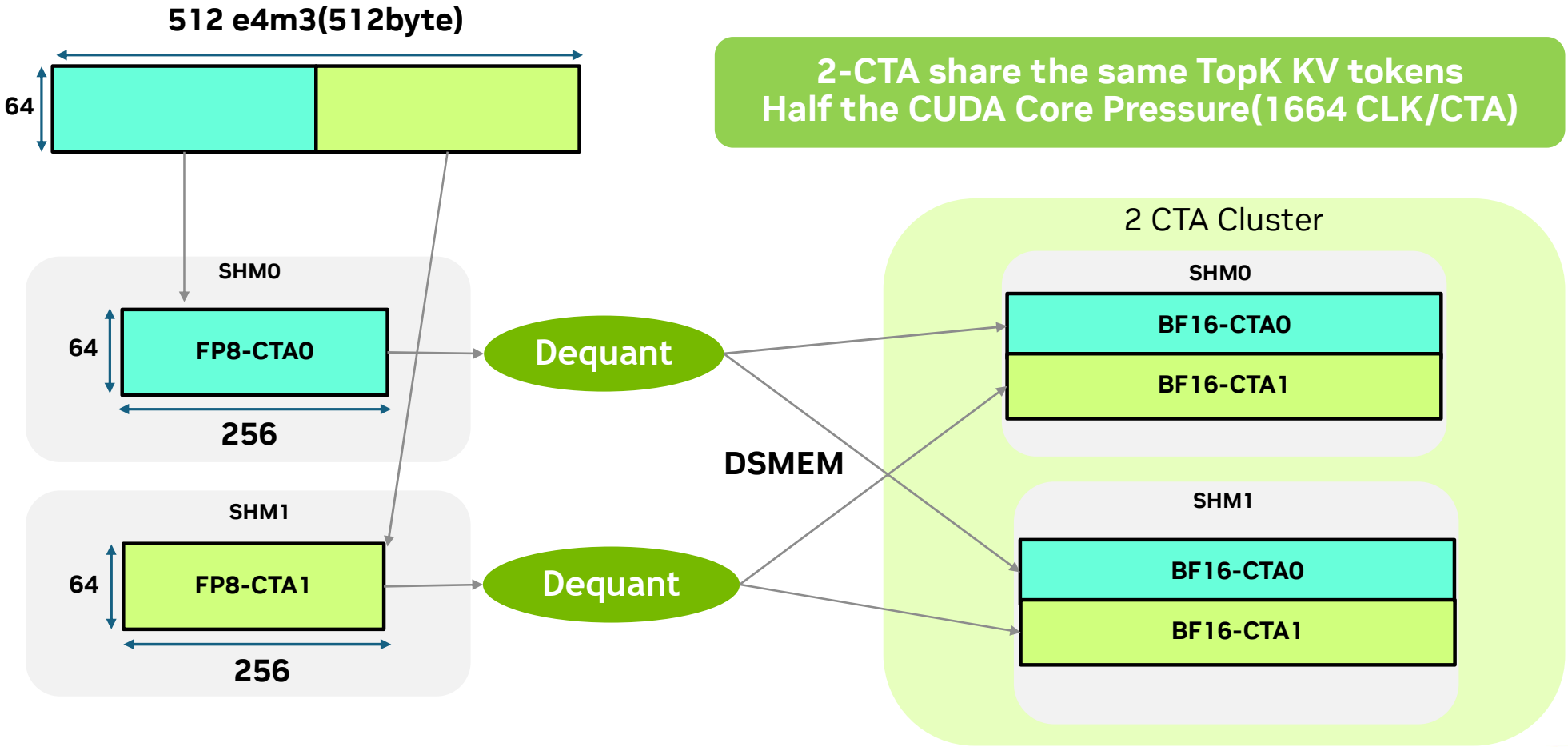
- Q_LEN = 1, or more(MTP is enabled)
  - Using splitKV to balance the SM workload
- FP8 KV Layout
  - Dequantization of FP8
    - FP8 -> FP16 (64 ops/clk/SM)
    - FP16 -> FP32 (64 ops/clk/SM)
    - FP32 -> BF16 (16 ops/clk/SM)
    - BF16 * BF16 Scale (128 ops/clk/SM on Blackwell)
  - Convert 64*512 FP8 to BF16 needs **3328 CLK/CTA!!**
  - GEMM:
    - BMM1(QK):2*64*64*576 = 4.6M ops/CTA
    - BMM2(SV): 2*64*64*512 = 4Mops/CTA
    - Tensorcor: 8192 ops/CLK/SM
    - Total CLK: **1088 CLK/CTA**



Solution: 2-CTA pair with DSMEM

2-CTA Dequantization in Sparse Decoding

Each CTA Does Half, Multi-cast through Distributed SMEM

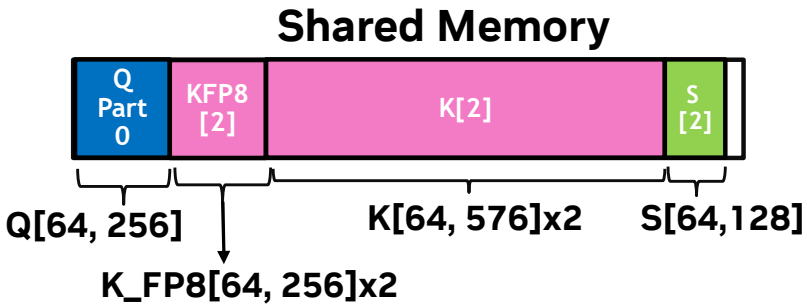


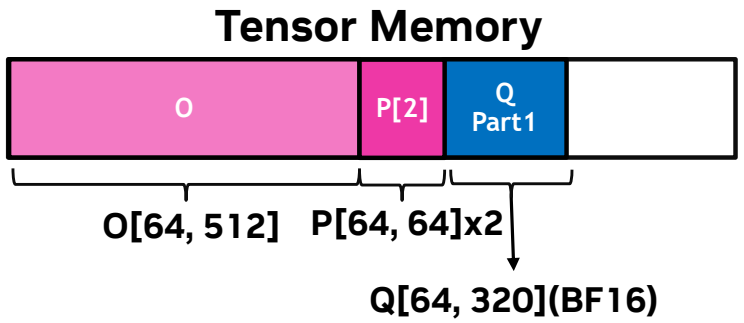
Not represent the real layout, just easy to draw

DSA Sparse Decoding

Memory Layout

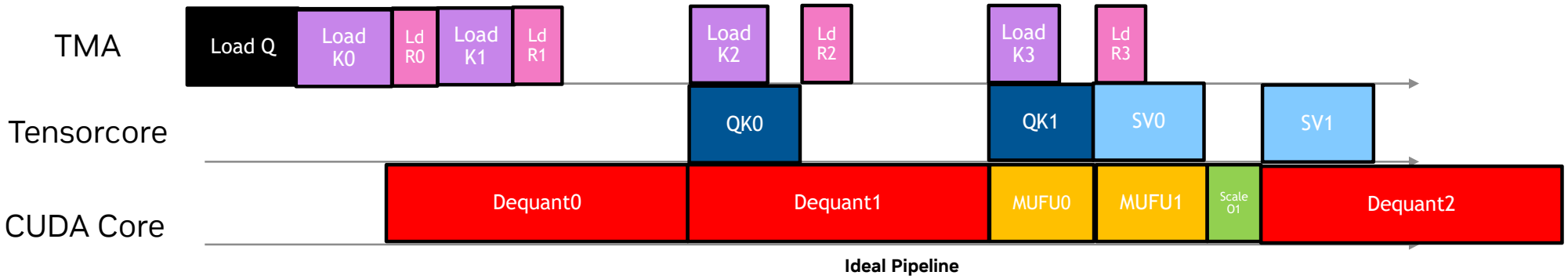
- Shared Memory(Double Buffer)
  - Q_Part0 (64*256 BF16 = **32KB**)
  - K_FP8[2] (64*256*2 FP8 = **32KB**)
  - K_BF16[2] (64*576*2 BF16 = **144KB**)
  - S[2] (64*64*2 BF16 = **16KB**)
  - Total: 224KB
- Tensor Memory:
  - P[2] = QK^T (64 * 64 * 2 FP32 = **32KB**)
  - O= SV (64 * 512 FP32 = **128KB**)
  - Q_ Part1 (64 * 320 BF16 = **40KB**)
  - Total: 200KB





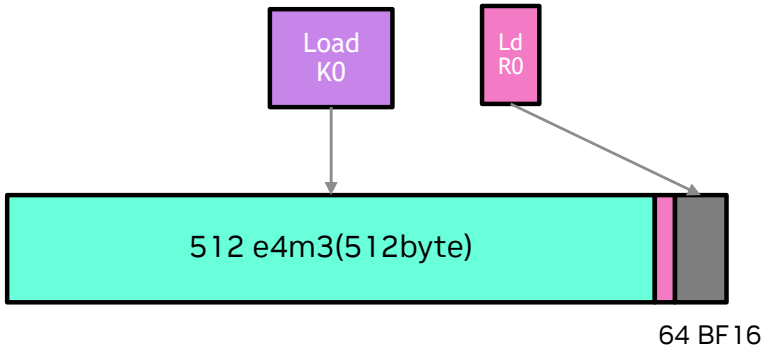
DSA Sparse Decoding

Pipeline



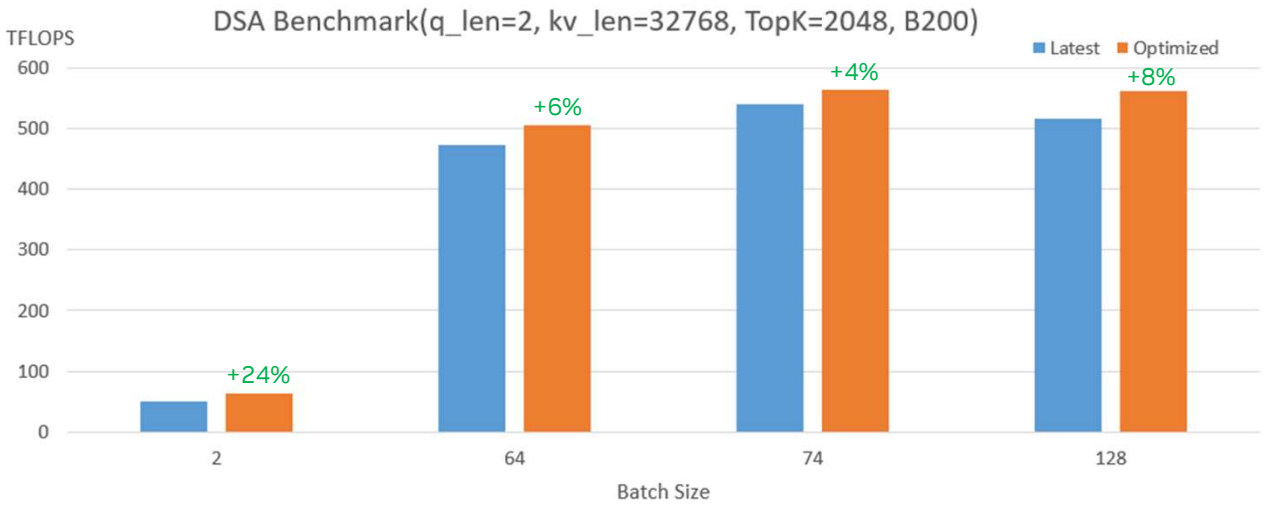
Ideal Pipeline

- CTA dim:[2, q_len, sm_parts]
- 384 threads(3 Warpgroup)
  - 128 for TMA
  - 128 for CUDA Core(Dequant & softmax & re-scale)
  - 32 for MMA, 32 for KV topK Index



Benchmark

Sparse Decoding Comparing with Naïve Approach

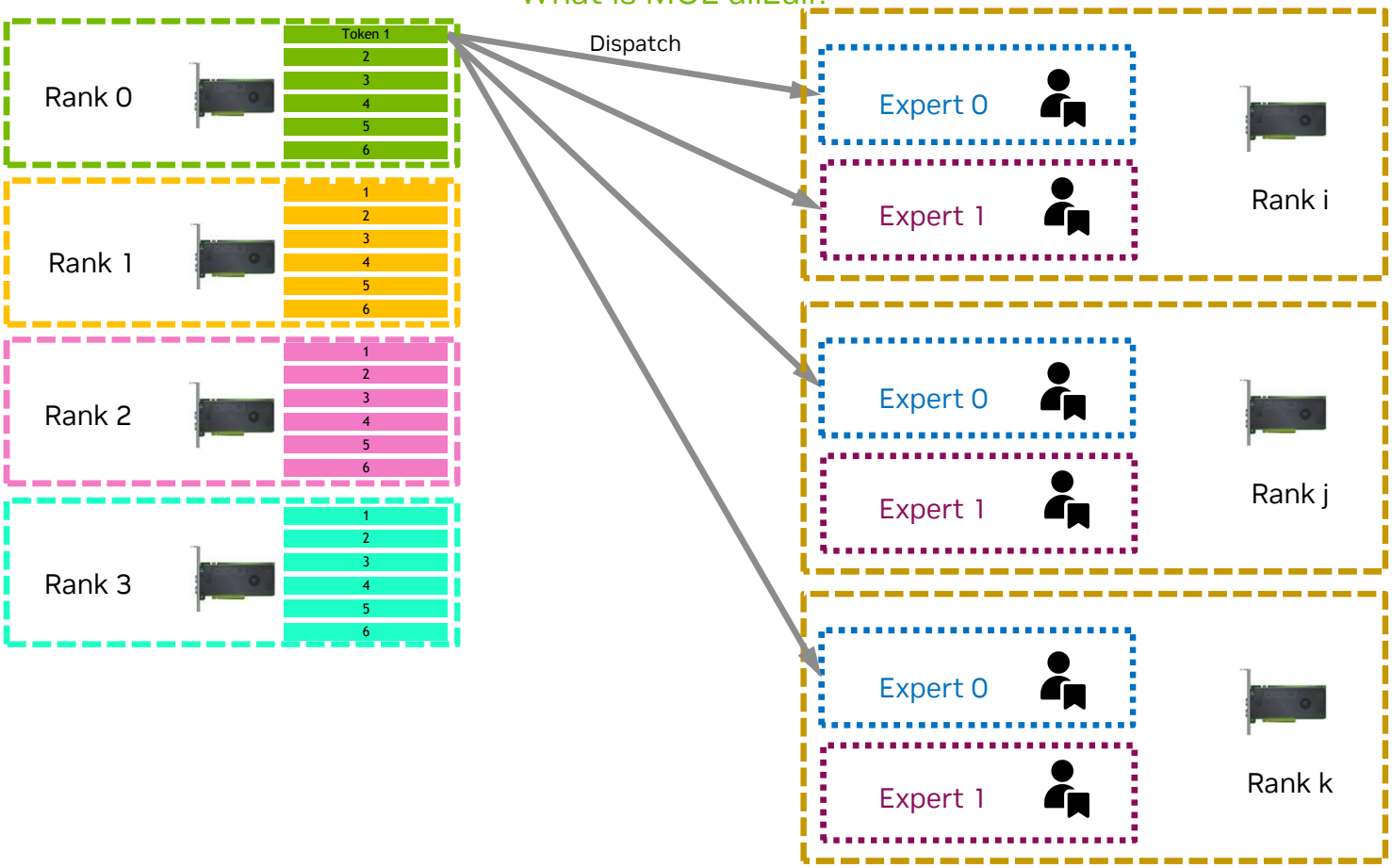


*For technical disscussion and reference only, perf. may vary based on different product portofolio.

Part 3. DeepEP

Background

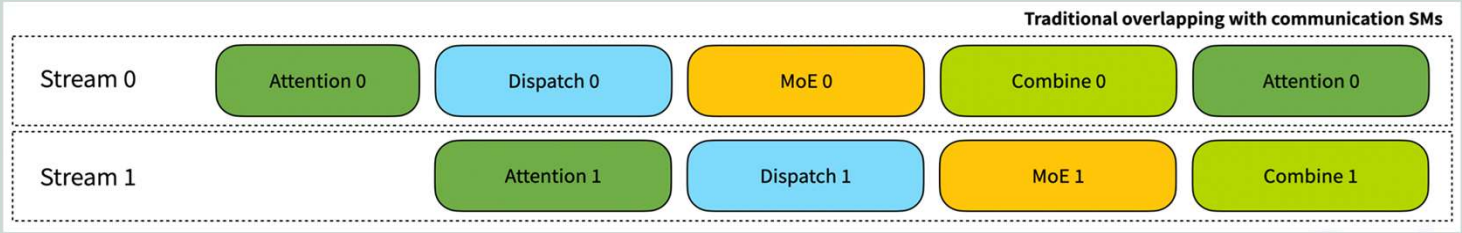
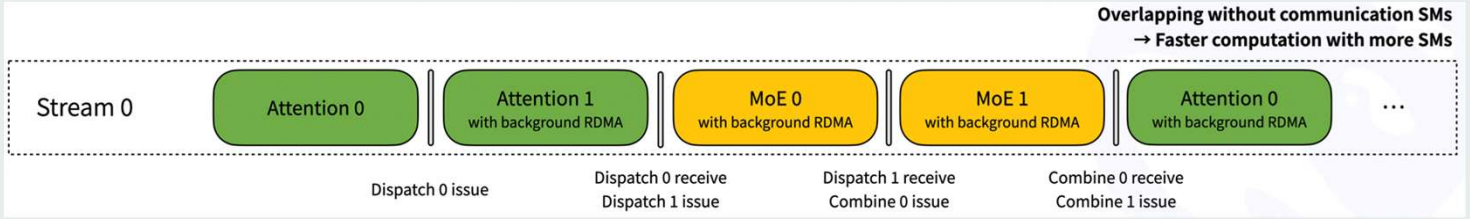
What is MOE all2all?



MOE all2all is a collective communication operation where tokens from every rank are simultaneously distributed to all other ranks based on their expert assignments.

Communication Kernels

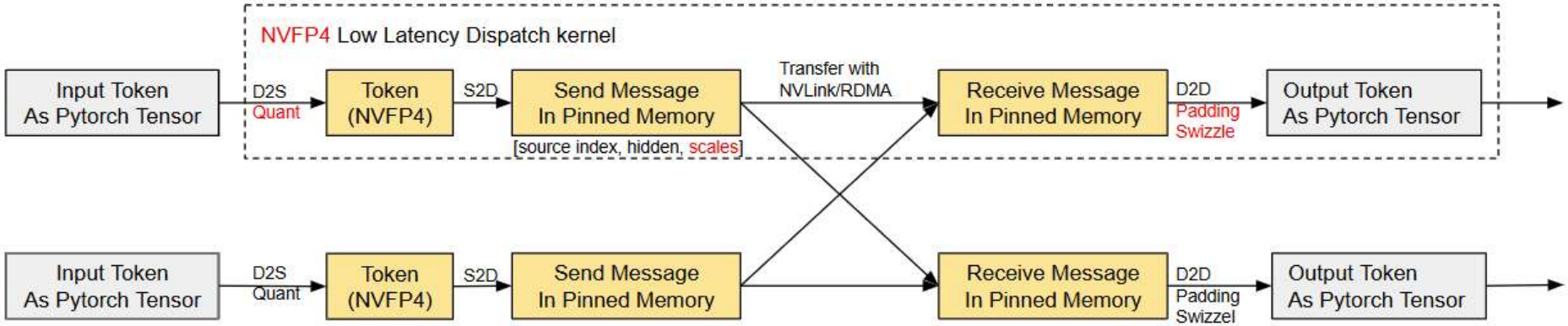
Normal Mode vs Low Latency Mode

Communication Kernel Mode	How To Overlap Computation and Communication
Normal Mode (Designed for Training)	
Low latency Mode (Designed for Inference Decode)	

Reference: [deepseek-ai/DeepEP: DeepEP: an efficient expert-parallel communication library]

Low precision dispatch

Low Latency Dispatch With NVFP4 Data Format



- Quant in Dispatch kernel
- Use global scale Instead of per tensor scale
- Padding and swizzle to adapt with GroupedGemm in FlashInfer

Reference: [Support nvfp4 low latency mode dispatch by shifangx · Pull Request #341 · deepseek-ai/DeepEP]

Low precision dispatch

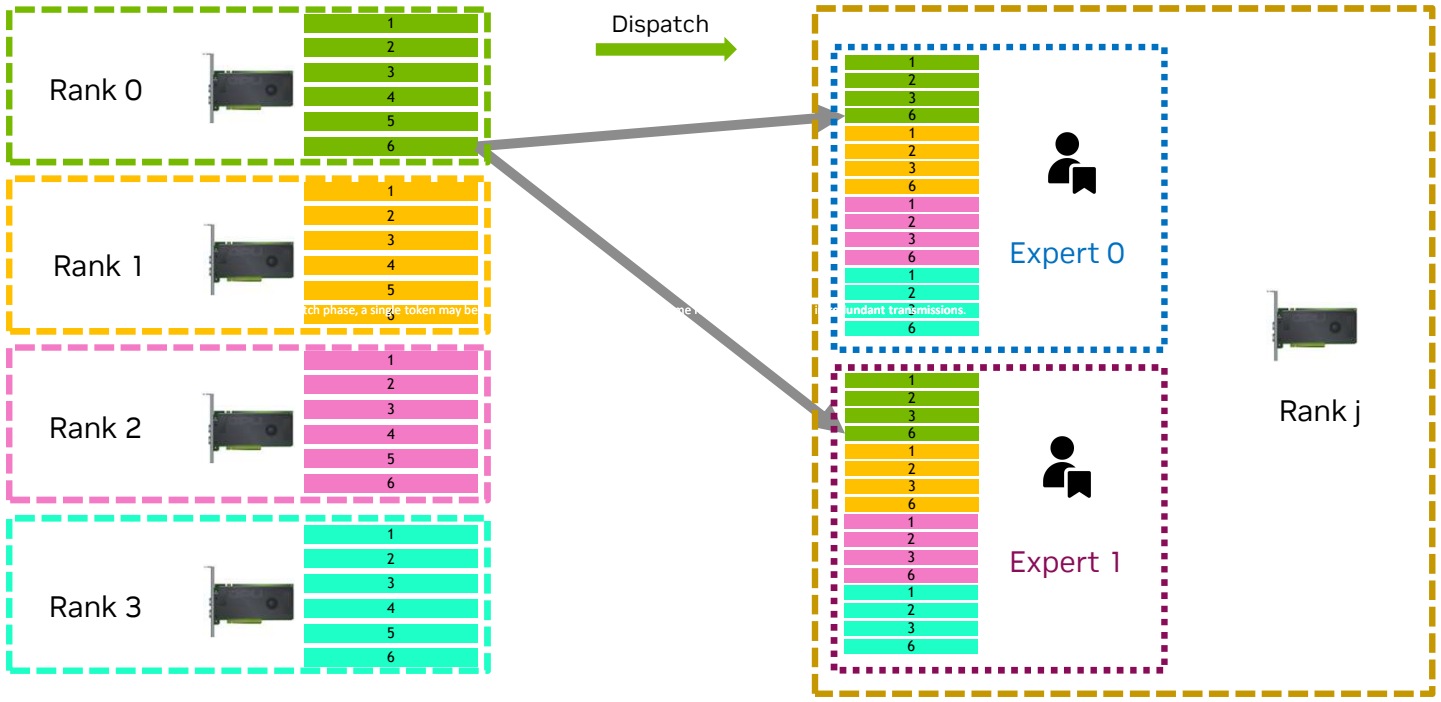
Performance Grain with NVFP4

	FP8	NVFP4	NVFP4 with larger batch size
Batch size	768	768	1024
Throughput	~9253 tok/s	~11000 tok/s	~12000 tok/s

Reference: how to reproduce these cases https://github.com/shifangx/Scripts-SGLang/tree/main

One-sided dispatch kernel and deduplication

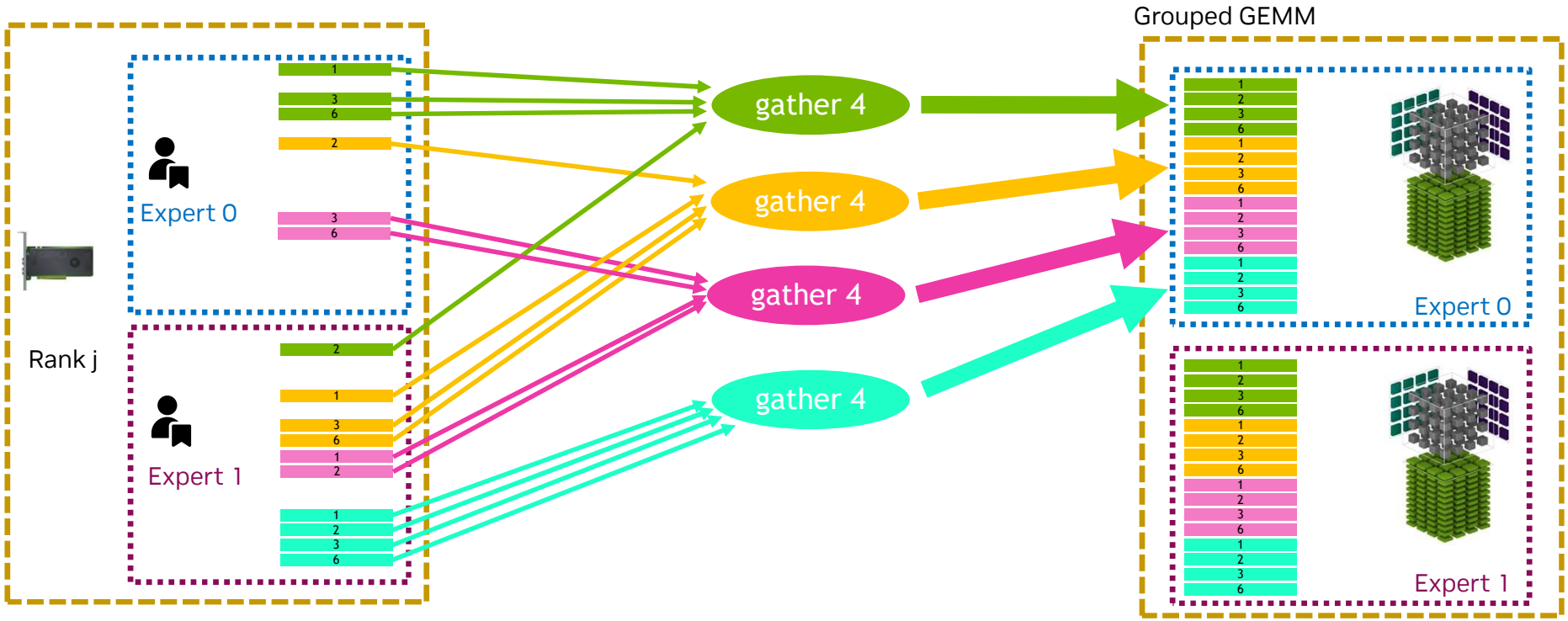
The one-sided all2all kernel in TRT-LLM



During the dispatch phase, a single token may be routed to multiple experts on the same rank, which can result in **redundant transmissions**.

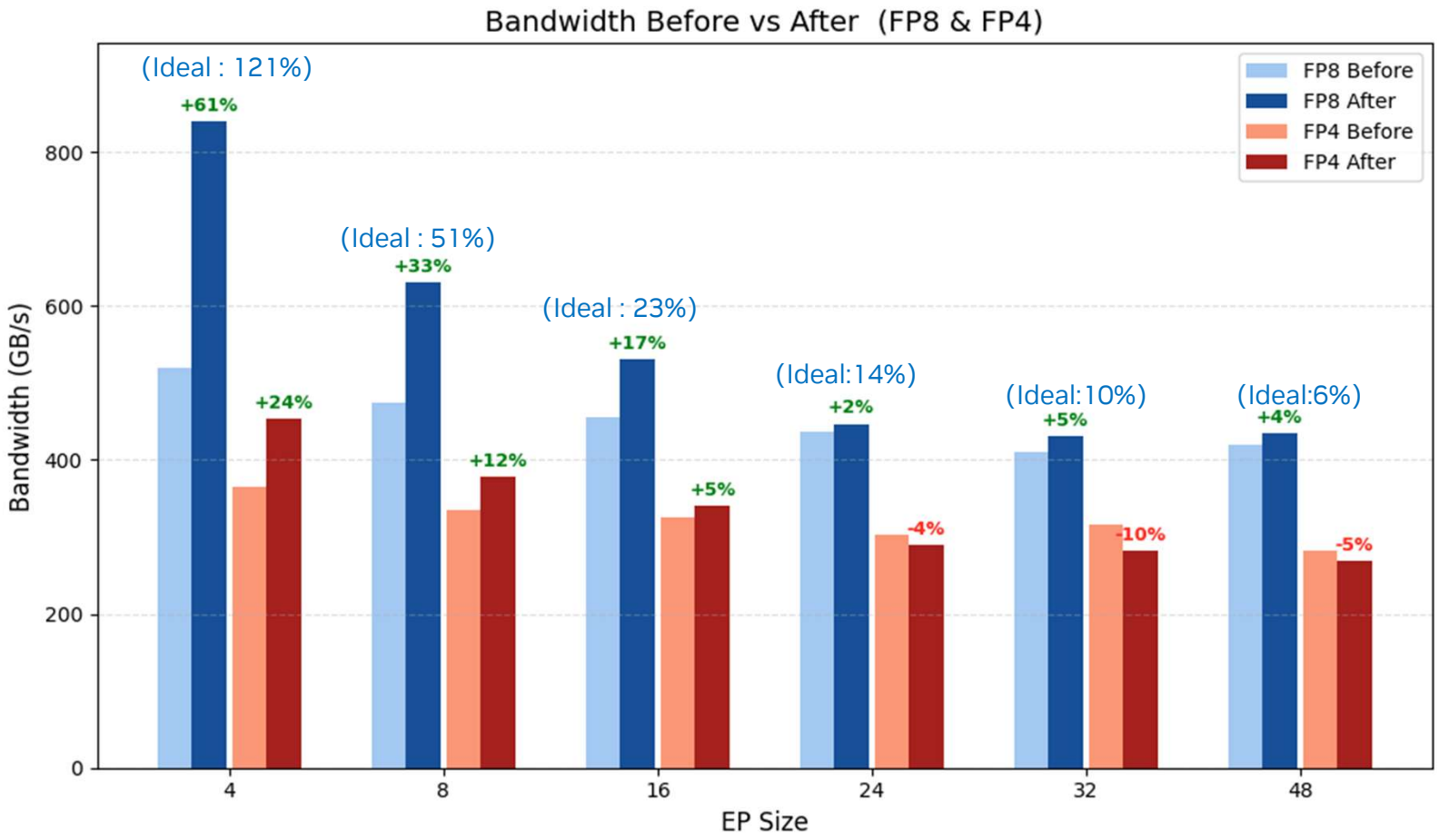
One-sided dispatch kernel and deduplication

Fuse expansion into MoE GEMM



One-sided dispatch kernel and deduplication

Performance gained



Summary

- DeepGEMM
  - MoE GEMM
  - V3.2 Indexer
- FlashMLA
  - DSA Kernel on Blackwell(Prefill & Decoding)
- DeepEP
  - MoE All-to-All with NVFP4

THANK YOU